UI Systems4

UI systems

Unity provides three UI systems: UI Toolkit, uGUI, and IMGUI. You can use these systems to create user interfaces (UI) for the Unity Editor and applications made in the Unity Editor.

Topic	Description
Comparison of UI systems in Unity	Unity intends for UI Toolkit to become the recommended UI system for new UI development projects, but it still lacks some features available in uGUI and IMGUI. These legacy systems are better suited for certain use cases and are required to support legacy projects. Choose a UI system based on the type of UI you need to develop and the features it requires.
UI Toolkit	The latest UI system, designed for optimized performance across platforms and inspired by standard web technologies. Use UI Toolkit to create extensions for the Unity Editor, and to create runtime UI for games and applications.
uGUI (Unity UI)	A legacy, GameObject-based UI system for developing runtime UI for games and applications. uGUI uses components and the Game view to arrange, position, and style the UI.
IMGUI (Immediate Mode GUI)	A legacy, code-driven UI system that uses the `OnGUI` function to draw and manage UI. Use IMGUI to create custom **Inspectors** for script components, extensions for the Unity Editor, and in-game debugging displays. It's not recommended for runtime UI.

Additional resources

Comparison of UI systems in Unity

Comparison of UI systems in Unity

UI Toolkit is intended to become the recommended UI system for you to develop UI in your projects. However, in its current release, UI Toolkit does not have some features that uGUI (Unity UI) and IMGUI (Immediate Mode GUI) support. uGUI and IMGUI are more appropriate for certain use cases, and are required to support legacy projects.

This page provides a high-level feature comparison of UI Toolkit, uGUI, and IMGUI, and their respective approaches to UI design.

General consideration

The following table lists the recommended and alternative system for runtime and Editor:

Unity 6	Recommendation	Alternative
Runtime	uGUI (Unity UI)	UI Toolkit
Editor	UI Toolkit	IMGUI

Roles and skill sets

Your team's skill set and comfort level with different technologies is also an important consideration.

The following table lists the recommended system for different roles:

Roles	UI Toolkit	uGUI (Unity UI)	IMGUI	Skill sets
Programmer	Yes	Yes	Yes	Programmers can use any game development tool or API.

Technical Artist	Partial	Yes	No	Technical artists who are familiar with Unity's GameObject-based tools and workflows are likely to be comfortable working with GameObjects, Components, and the Scene view. They might not be comfortable with UI Toolkit's web-like approach or IMGUI's pure C# approach.
UI Designer	Yes	Partial	No	UI designers who are familiar with UI creation tools are likely to be comfortable with UI Toolkit's document-based approach and can use the UI Builder to visually edit their UI. If they are not familiar with GameObject-based workflows, they might

				require help from programmers or level designers.

Innovation and development

UI Toolkit is in active development and releases new features frequently. uGUI and IMGUI are established and production-proven UI systems that are updated infrequently.

uGUI and IMGUI might be better choices if you need features that are not yet available in UI Toolkit, or you need to support or reuse older UI content.

Runtime

UI Toolkit is an alternative to uGUI (Unity UI) if you create a screen overlay UI that runs on a wide variety of screen resolutions. Consider UI Toolkit to do the following:

Produce work with a significant amount of user interfaces
Require familiar authoring workflows for artists and designers
Seek textureless UI rendering capabilities

uGUI is the recommended solution for the following:

UI positioned and lit in a 3D world
VFX with custom **shaders**

and materials
Easy referencing from `MonoBehaviours`

Use Cases

The following table lists the recommended system for major runtime use cases:

Unity 6	Recommendation
Multi-resolution menus and HUD in intensive UI projects	UI Toolkit
World space UI and VR	uGUI

UI that requires customized shaders and materials	uGUI

In details

The following table lists the recommended system for detailed runtime features:

Unity 6	UI Toolkit	uGUI
WYSIWYG authoring	Yes	Yes
Nesting reusable components	Yes	Yes
Global style management	Yes	No
Layout and Styling Debugger	Yes	Yes
Scene integration	Yes	Yes
Rich text tags	Yes	Yes*
Scalable text	Yes	Yes*
Font fallbacks	Yes	Yes*
Adaptive layout	Yes	Yes
Input system support	Yes	Yes
Serialized events	No	Yes

Visual Scripting support	No	Yes
Compatible with Rendering pipelines	Yes	Yes
Screen-space (2D) rendering	Yes	Yes
World-space (3D) rendering	No	Yes
Custom materials and shaders	No	Yes
Sprites / Sprite atlas support	Yes	Yes
Dynamic texture atlas	Yes	No
Textureless elements	Yes	No
UI anti-aliasing	Yes	No
Rectangle clipping	Yes	Yes
Mask clipping	No	Yes
Nested masking	Yes	Yes
UI transition animations	Yes	No
Integration with Animation Clips	No	Yes

and Timeline		

*Requires the TextMesh Pro package

Editor

UI Toolkit is recommended if you create complex editor tools. UI Toolkit is also recommended for the following reasons:

Better reusability and decoupling
Visual tools for authoring UI
Better scalability for code maintenance and performance

IMGUI is an alternative to UI Toolkit for the following:

Unrestricted access to editor extensible capabilities
Light API to quickly render UI on screen

Use Cases

The following table lists the recommended system for major Editor use cases:

Unity 6	Recommendation
Complex editor tool	UI Toolkit
Property drawers	UI Toolkit
Collaboration with designers	UI Toolkit

In details

The following table lists the recommended system for detailed Editor features:

Unity 6	UI Toolkit	IMGUI
WYSIWYG authoring	Yes	No
Nesting reusable components	Yes	No

Global style management	Yes	Yes
Layout and Styling Debugger	Yes	No
Rich text tags	Yes	Yes
Scalable text	Yes	No
Font fallbacks	Yes	Yes
Adaptive layout	Yes	Yes
Default Inspectors	Yes	Yes
Inspector: Edit custom object types	Yes	Yes
Inspector: Edit custom property types	Yes	Yes
Inspector: Mixed values (multi-editing) support	Yes	Yes
Array and list-view control	Yes	Yes
Data binding: Serialized properties	Yes	Yes

Additional resources

Migrate from uGUI to UI Toolkit
Migrate from IMGUI to UI Toolkit

UI Toolkit

UI Toolkit

UI Toolkit is a collection of features, resources, and tools for developing user interface (UI).

Topic	Description
Introduction to UI Toolkit	Learn about UI Toolkit, its features, roles, and responsibilities in the development process.
Get started with UI Toolkit	Follow a simple workflow to get started with UI Toolkit.
UI Builder	Use a visual authoring tool to create and edit UI Toolkit assets such as UXML and USS files.
Structure UI	Structure your UI with either UXML or C#.
Style UI	Style your UI with USS.
Control behavior with events	Map user interactions to elements, such as input, touch and pointer interactions, drag-and-drop operations, and other event types. This system includes a dispatcher, a handler, a synthesizer, and a library of event types.
UI Renderer	Use a rendering system built directly on top of Unity's graphics device layer.
Data binding	Link properties to the controls that modify their values.
Support for Editor UI	A set of components to create Editor UI.
Support for runtime UI	A set of components to create runtime UI.
Work with text	Learn how to work with text in UI Toolkit, including fonts, styles, and text elements.
Test UI	Learn how to test and debug your UI.

UI Toolkit examples	A collection of UI Toolkit examples to help you learn how to use it.
Migration guides	Guides to help you migrate from uGUI or IMGUI to UI Toolkit.

Additional resources

- 📚 **Documentation**: Comparison of UI systems in Unity
- 📺 **Video**: Unity Input System in Unity 6 (4/7): Input System and UI toolkit
- 📖 **E-Book**: Create scalable and performant UI with UI Toolkit in Unity 6
- ⚙️ **Sample project**: Dragon Crashers - UI Toolkit Sample project
- ⚙️ **Sample project**: QuizU - A UI toolkit sample

Introduction to UI Toolkit

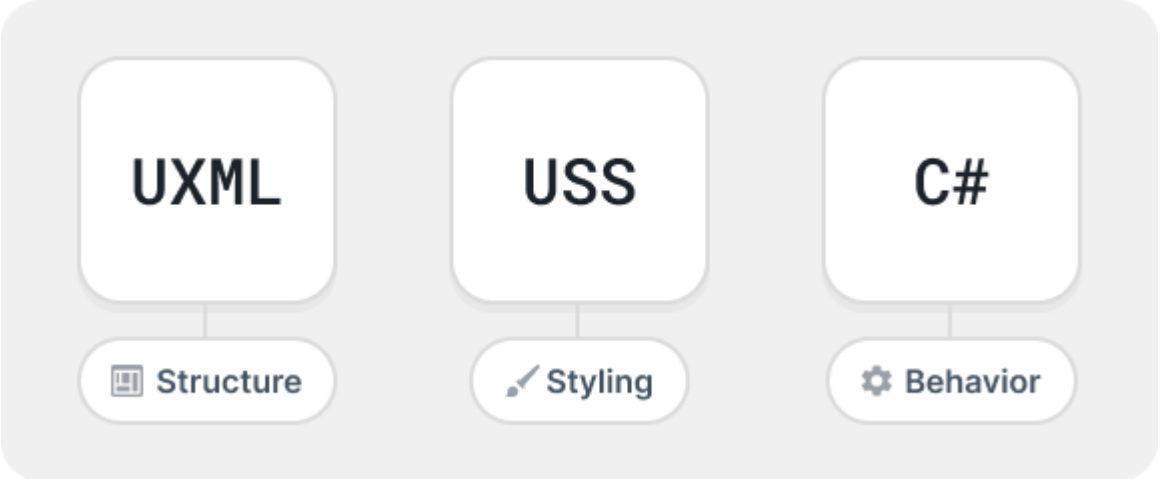
Introduction to UI Toolkit

You can use UI Toolkit to develop custom UI and extensions for the Unity Editor, and runtime UI for games and applications.

Web-inspired design

UI Toolkit is designed for optimal performance and adaptability across platforms and it is inspired by standard web technologies. If you have experience developing web pages or applications, the core concepts of UI Toolkit might be familiar to you. With UI Toolkit, you can separate the structure, style, and behavior of your UI, similar to how you might with HTML, CSS, and JavaScript.

UI Toolkit uses the following assets, similar to web development:



Relationship between UI assets and C# code

UXML

UXML is a markup language inspired by HTML and XML that defines UI structure and reusable templates. While you can build UI in C#, using UXML is recommended.

USS

USS are style sheets that apply visual styles and layout rules to UI. Similar to CSS, USS supports a subset of standard CSS properties. While you can define styles in C#, using USS is recommended.

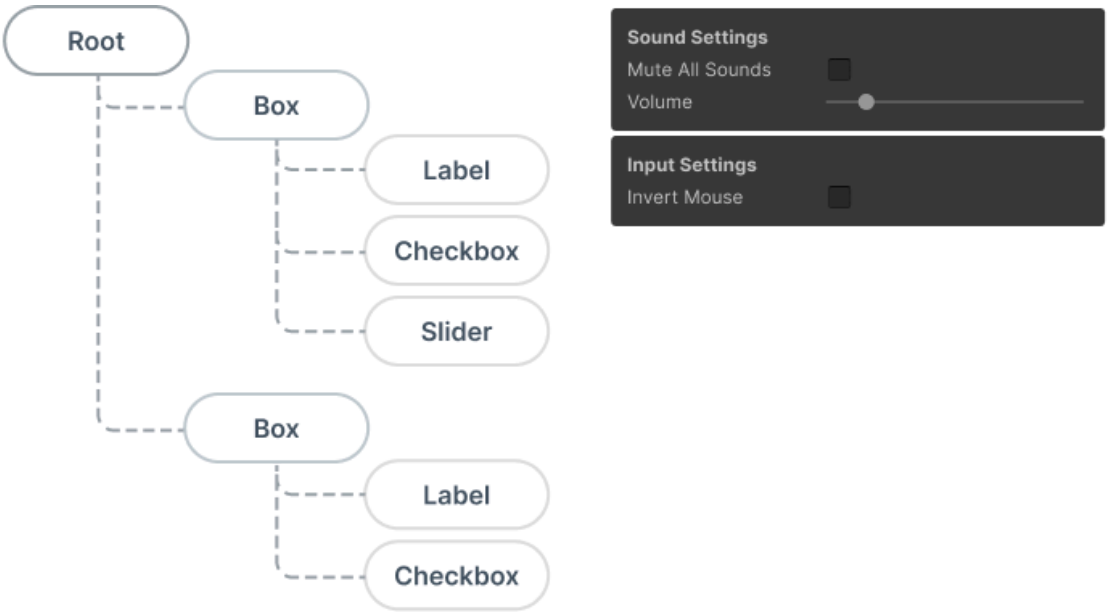
Retained mode architecture

UI Toolkit operates in a retained mode architecture, which means the UI builds a hierarchical structure and maintains it in memory. The framework handles rendering and updates to the UI based on the state of that tree. Here's what retained mode means in detail:

Declarative structure

UI components are defined and managed as objects or nodes within a visual tree

. Visual tree is an object graph made of lightweight nodes that holds all the elements in a window or panel. It defines every UI built with UI Toolkit. You declare the UI structure once, UI Toolkit takes care of rendering and updates based on the changes to the tree.



Visual tree example

State persistence

The UI elements in retained mode persist in memory after creation. You can modify their properties, and UI Toolkit reflects changes in the next rendering cycle. For example, if you have a button, you don't need to redraw it manually. Updating its properties, such as position or appearance, is enough. The system handles the rest.

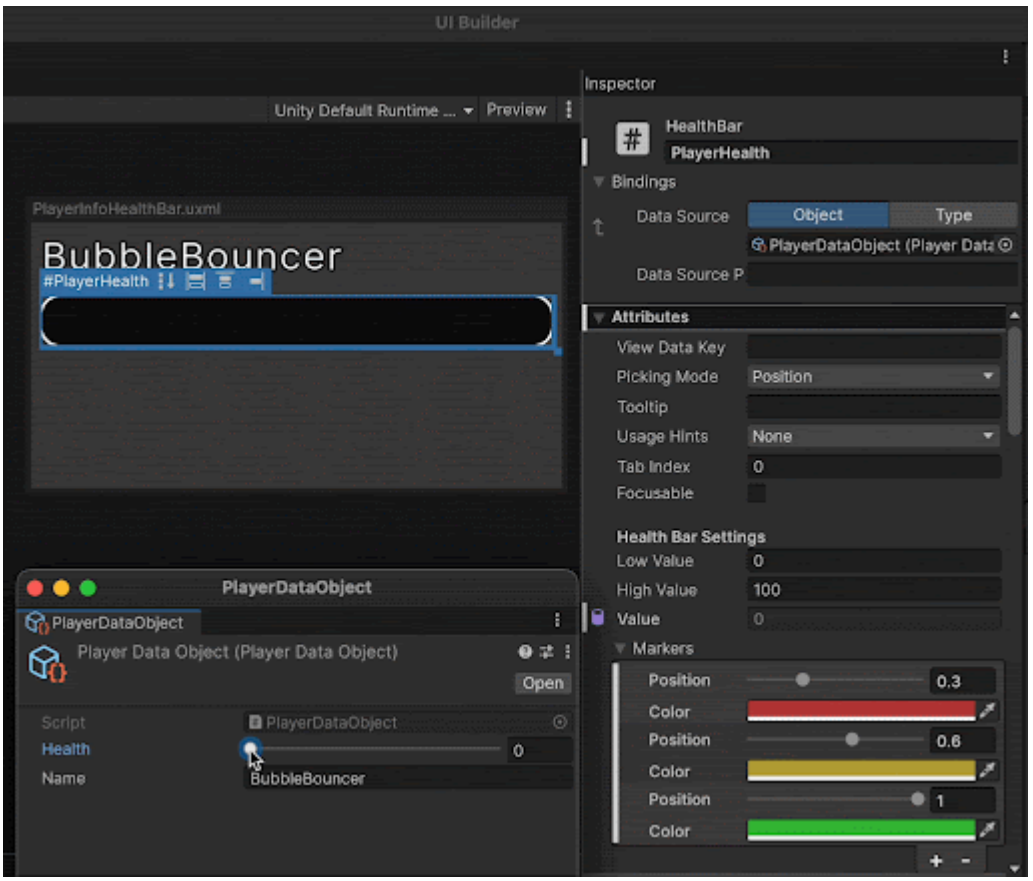
Event handling

Events such as clicks, drags, or key presses are routed automatically to the appropriate **visual elements**

in the tree. You attach handlers to UI elements, UI Toolkit propagates events appropriately.

Data binding

UI Toolkit supports a data binding system that allows you to link properties of UI elements to data sources. This means that when the data changes, the UI updates automatically, and vice versa. This is similar to how frameworks like React or Angular handle data binding in web development.

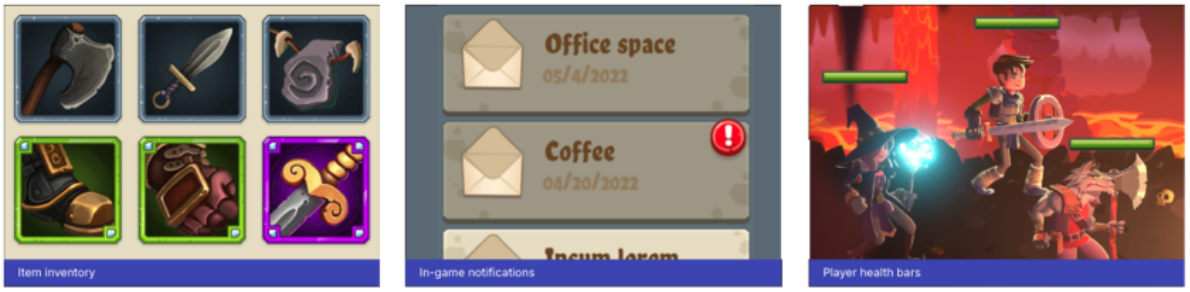


Data binding example

Layout engine

UI Toolkit uses a layout engine based on the CSS Flexbox model. This lets you to define how UI elements are positioned and sized within their parent containers. The layout engine automatically calculates the size and position of elements based on their properties, making it easier to create responsive designs.

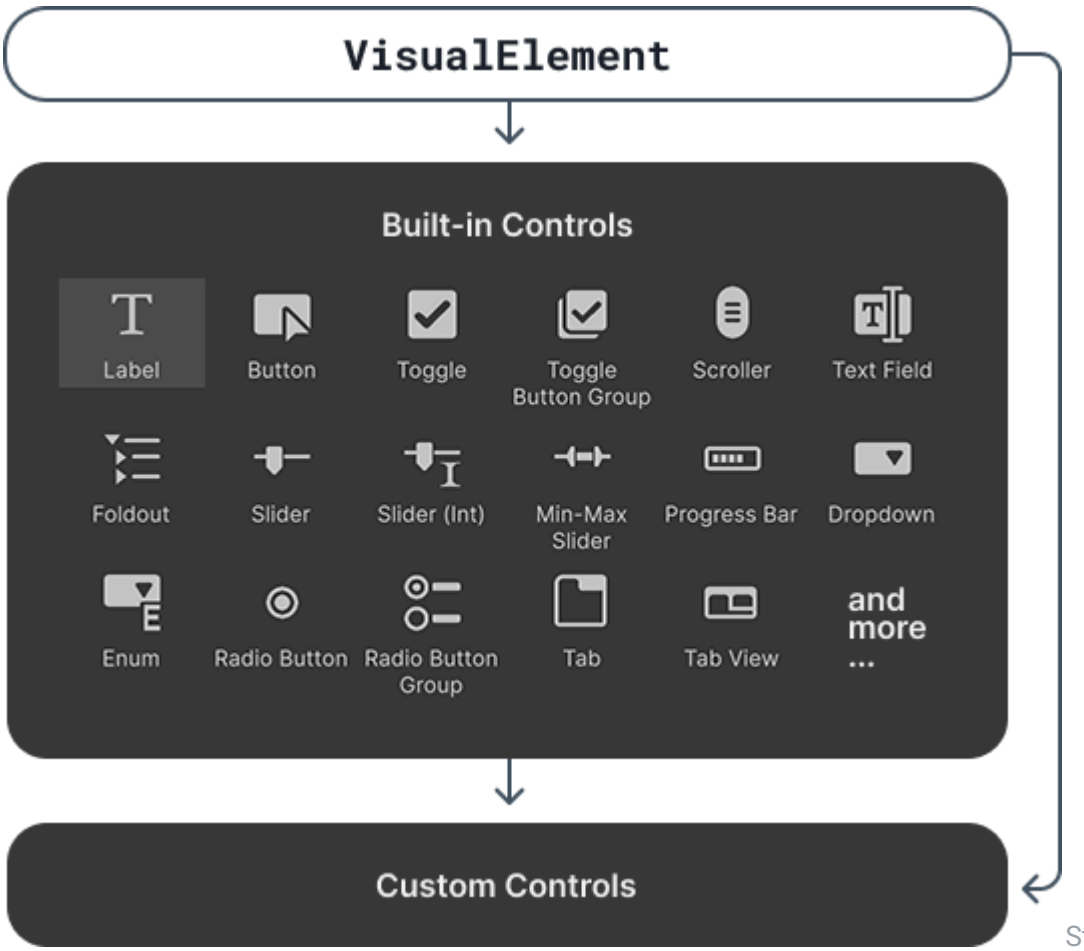
Use flex properties for efficient space distribution, easy reordering, and absolute positioning of elements. The layout engine also supports alignment, justification, wrapping, and much more.



Flexbox layout examples

Library of UI controls

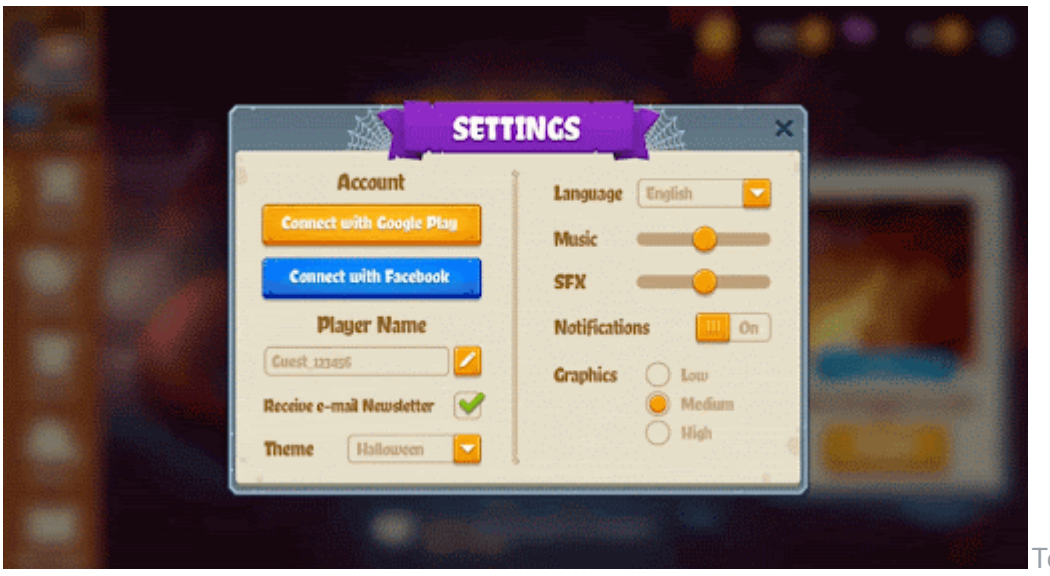
UI Toolkit provides a rich library of standard UI controls that you can use to build your interfaces, such as buttons, toggles, and list and tree views. You can use them as is, customize them, or create your own controls.



Standard UI controls

Text and fonts

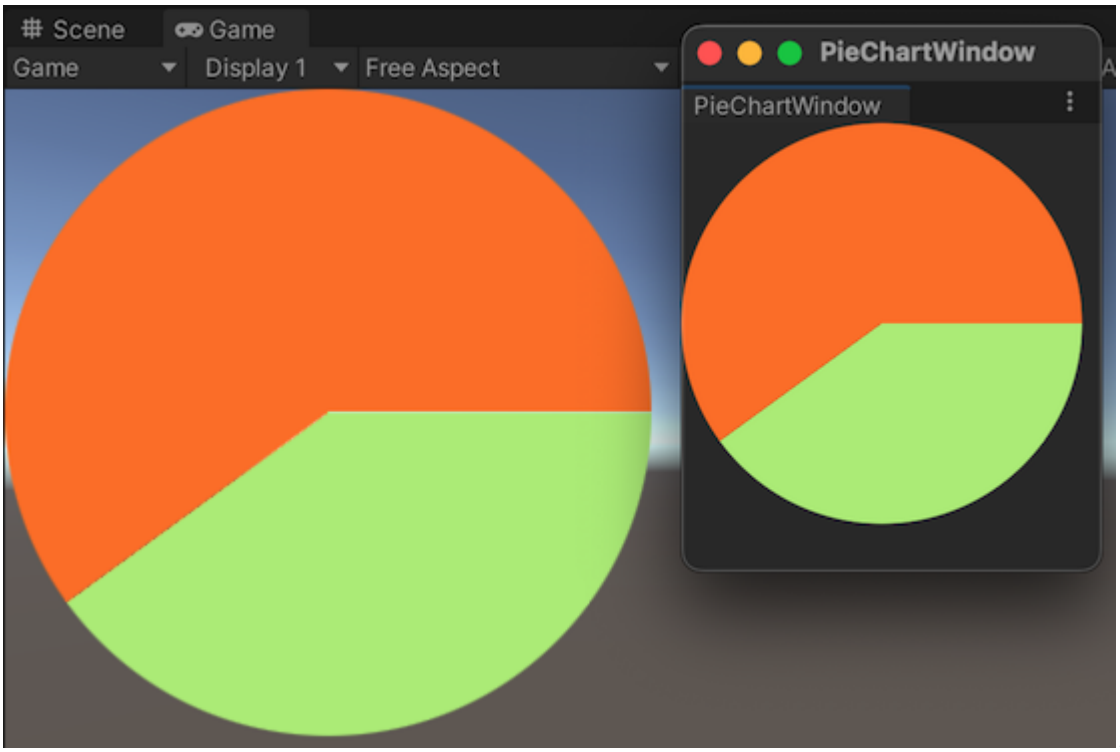
UI Toolkit supports a text system that allows you to work with fonts, styles, and text elements. You can use system fonts or import custom fonts, and apply styles such as bold, italic, or underline. The text system is designed to work seamlessly with the layout engine, ensuring that text flows correctly within your UI.



Text and fonts example

Support for Editor and runtime UI

UI Toolkit lets you create a UI that works seamlessly in both Editor and runtime UI. The same UI elements, styling, and layout system apply across both scenarios, reducing duplication and ensuring consistency. Whether you're building in-game menus, heads-up displays (HUDs), or custom Editor tools, UI Toolkit provides the capability to adapt your UI for different contexts.



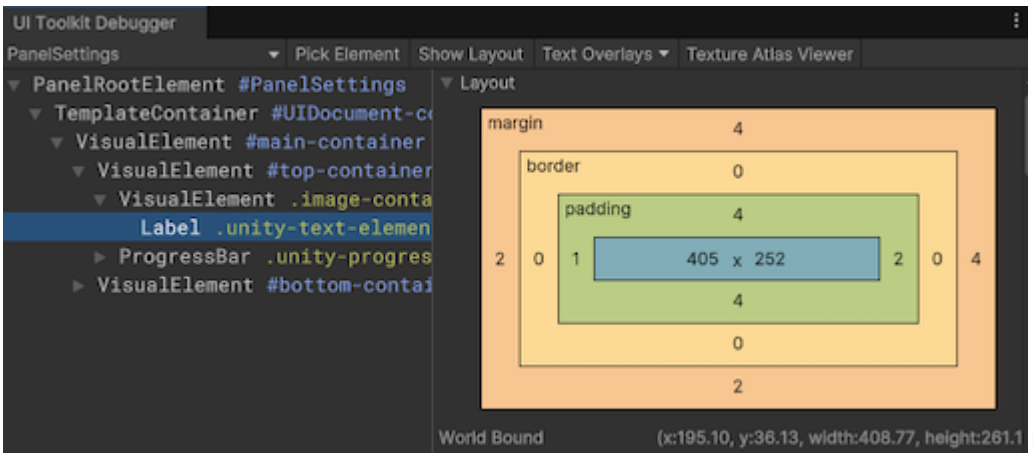
A pie chart appears in both the scene and an Editor window.

UI tools and resources

UI Toolkit provides a set of tools and resources to help you design, develop, and debug your UI.

UI Debugger

UI Debugger is a diagnostic tool that resembles a web browser's debugging view. Use it to explore a hierarchy of elements and get information about their underlying UXML structure and USS styles.



UI Debugger example

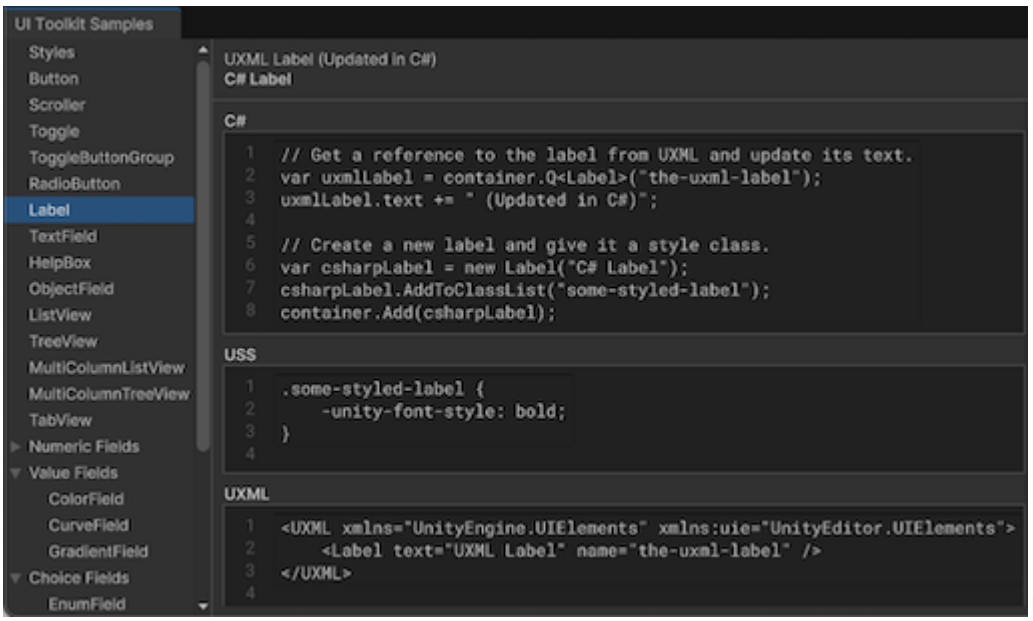
UI Builder

UI Builder is a visual authoring tool that lets you create and edit UXML and USS files. You can design your UI visually, drag and drop elements, and customize styles without writing code.

UI Builder example

UI samples

A library of code samples for UI controls that you can view in the Editor under **Window** > **UI Toolkit** > **Samples**.



UI Toolkit samples

Roles and responsibilities

In a studio, UI Toolkit can be used across different disciplines, with each role contributing to specific parts of the process.

UX/UI Designers

UX/UI designers are responsible for the overall look and feel of the UI. They create wireframes, mockups, and prototypes to define the user experience.

Designers use UI Builder to visually design the UI and create UXML and USS files for structure and styling.

Developers

Developers implement the behavior of the UI and integrate it with game or editor logic. They work closely with designers to ensure that the UI meets the design specifications and functions correctly.

Developers use C# to create custom functionality, handle user interactions, and bind data. They also use UI Toolkit's API to interact with the UI controls created by designers and to develop custom UI controls that designers can use and customize in their designs.

Technical Artists

Technical artists often act as a bridge between designers and developers. They ensure that the visual assets are optimized for performance and integrate well with the UI.

Technical artists assist in creating USS styles and troubleshooting issues that arise during development.

QA/Testing

QA/testers test the UI to ensure it functions correctly and meets design specifications.

Testers use the UI Debugger to inspect the UI hierarchy and verify interactions.

Additional resources

Get started with UI Toolkit
UI Toolkit examples

Get started with UI Toolkit

Get started with UI Toolkit

Want to create your first UI with UI Toolkit? Use this basic UI Toolkit workflow example to get started.

Note: For demonstration purposes, this guide describes how to add UI controls for the Editor UI. However, the instructions on adding UI controls to a UI Document also apply to runtime UI. For more information, refer to Get started with runtime UI.

If you perform a specific task often, you can use UI Toolkit to create a dedicated UI for it. For example, you can create a custom Editor window. The example demonstrates how to create a custom Editor window and add UI controls into your custom Editor window with UI Builder, UXML, and C# script.

You can find the completed files that this example creates in this GitHub repository.

Create a custom Editor window

Create a custom Editor window with two labels.

1. Create a project in Unity Editor with any template.
2. In the **Project** window, right-click in the `Assets` folder, and then select **Create** > **UI Toolkit** > **Editor Window**.
3. In **UI Toolkit Editor Window Creator**, enter `SimpleCustomEditor` in the **C#** box.
4. Keep the **UXML** checkbox selected and clear the **USS** checkbox.
5. Select **Confirm**.
6. To open the Editor window, select **Window** > **UI Toolkit** > **SimpleCustomEditor**.

You can find the source files for it in the `Assets/Editor` folder.

Add UI controls to the window

You can add UI controls to your window in the following ways:

Use the UI Builder to visually add the UI controls
Use an XML-like text file (UXML) to add the UI controls
Use C# script to add the UI controls

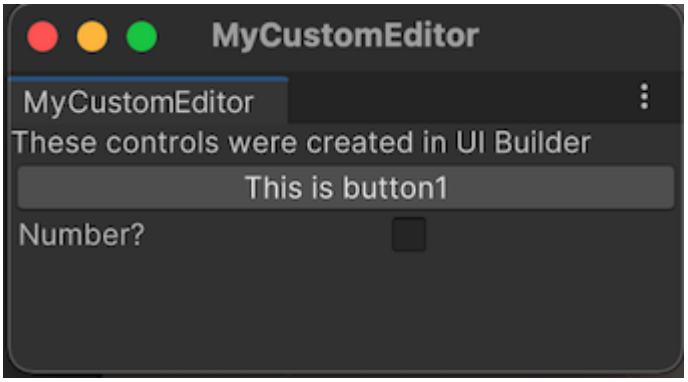
You can use any of these methods individually, or combine. The following examples create three sets of labels, buttons, and toggles with a combination of these methods.

Use UI Builder to add UI controls

To visually add UI controls to your window, use UI Builder. The following steps add a button and a toggle into your custom Editor window in addition to the default label.

1. In the `Editor` folder, double-click `SimpleCustomEditor.uxml` to open the UI Builder.
2. In the UI Builder, drag **Button** and **Toggle** from **Library** > **Controls** into the **Hierarchy** or the window preview in the **Viewport**.
3. In the **Hierarchy** window, select **Label**.
4. In the **Inspector**

6. window, change the default text to `These controls were created in UI Builder` in the **Text** field.
7. In the **Hierarchy** window, select **Button**.
8. In the **Inspector** window, enter `This is button1` in the **Text** field.
9. Enter `button1` in the **Name** field.
10. In the **Hierarchy** window, select **Toggle**.
11. In the **Inspector** window, enter `Number?` in the **Label** field.
12. Enter `toggle1` in the **Name** field.
13. Save and close the UI Builder window.
14. Close your custom Editor window if you haven't done so.
15. In the **Project** window, select `SimpleCustomEditor.cs` and make sure the **Visual Tree Asset** is set to `SimpleCustomEditor (Visual Tree Asset)` in the **Inspector** window.
16. Select **Window** > **UI Toolkit** > **SimpleCustomEditor** to re-open your custom Editor window to see the button and the toggle you just added.



Custom Editor Window with one set UI Controls

Use UXML to add UI controls

If you prefer to define your UI in a text file, you can edit the UXML to add the UI controls. The following steps add another set of label, button, and toggle into your window.

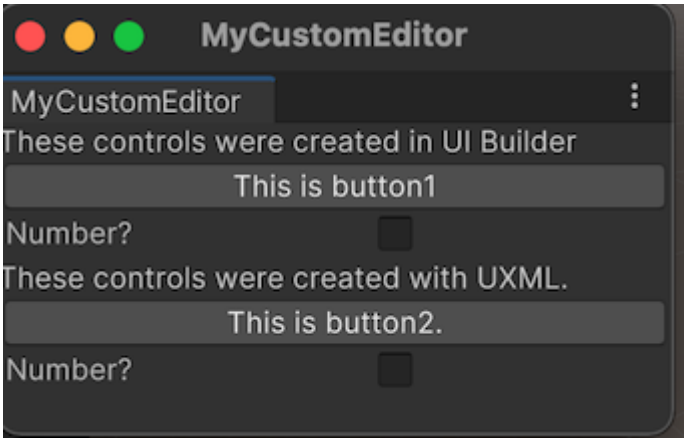
1. In the `Editor` folder, select **Assets** > **Create** > **UI Toolkit** > **UI Document** to create a UXML file called `SimpleCustomEditor_UXML.uxml`.
2. Select the arrow on `SimpleCustomEditor_UXML.uxml` in the **Project** window.
3. Double-click `inlineStyle` to open `SimpleCustomEditor_UXML.uxml` in a text editor.
4. Replace the contents of `SimpleCustomEditor_uxml.uxml` with the following:

```
  <ui:UXML xmlns:ui="UnityEngine.UIElements"
xmlns:uie="UnityEditor.UIElements"
xsi="http://www.w3.org/2001/XMLSchema-instance"
engine="UnityEngine.UIElements" editor="UnityEditor.UIElements"
noNamespaceSchemaLocation="../../UIElementsSchema/UIElements.xsd"
editor-extension-mode="False">
        <ui:Label text="These controls were created with UXML." />
        <ui:Button text="This is button2" name="button2"/>
        <ui:Toggle label="Number?" name="toggle2"/>
    </ui:UXML>
```

1. Open `SimpleCustomEditor.cs`.
2. Import the UXML file you created manually by adding the following to the `CreateGUI` method:

```
// Import UXML created manually.
var visualTree =
AssetDatabase.LoadAssetAtPath<VisualTreeAsset>("Assets/Editor/SimpleCustomEdi
tor_uxml.uxml");
VisualElement labelFromUXML = visualTree.Instantiate();
root.Add(labelFromUXML);
```

3.
4. Save your changes.
5. Select **Window** > **UI Toolkit** > **SimpleCustomEditor**. This opens your custom Editor window with three labels, two buttons, and two toggles.



Custom Editor Window with two sets UI Controls

Use C# script to add UI controls

If you prefer coding, you can add UI Controls to your window with a C# script. The following steps add another set of label, button, and toggle into your window.

1. Open `SimpleCustomEditor.cs`.
2. Unity uses `UnityEngine.UIElements` for basic UI controls like label, button, and toggle. To work with UI controls, you must add the following declaration if it's not already present.

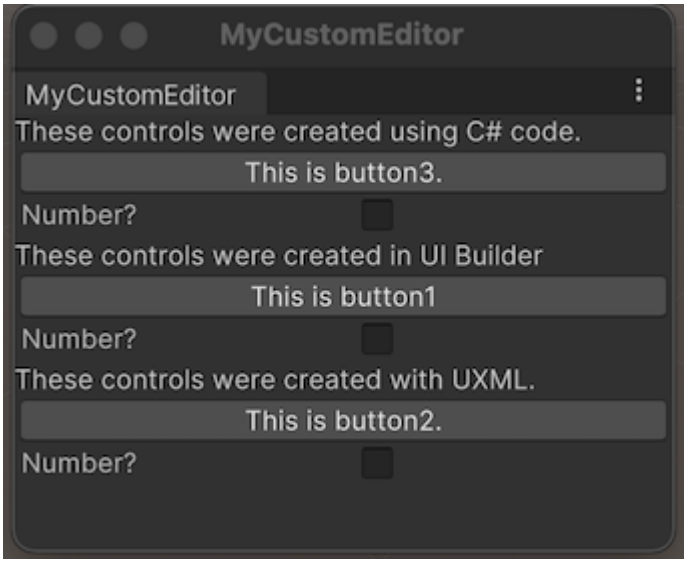
```
using UnityEngine.UIElements;
```

3.
4. Change the text of the existing label from `"Hello World! From C#"` to `"These controls were created using C# code."`.

5. The EditorWindow class has a property called `rootVisualElement`. To add the UI controls to your window, first instantiate the element class with some attributes, and then use the `Add` methods of the `rootVisualElement`.
   Your finished `CreateGUI()` method looks like the following:

```
public void CreateGUI()
{
    // Each editor window contains a root VisualElement object
    VisualElement root = rootVisualElement;

    // VisualElements objects can contain other VisualElements following a
tree hierarchy.
    Label label = new Label("These controls were created using C# code.");
    root.Add(label);

    Button button = new Button();
    button.name = "button3";
    button.text = "This is button3.";
    root.Add(button);

    Toggle toggle = new Toggle();
    toggle.name = "toggle3";
    toggle.label = "Number?";
    root.Add(toggle);

    // Instantiate UXML created automatically which is set as the default
VisualTreeAsset.
    root.Add(m_VisualTreeAsset.Instantiate());

    // Import UXML created manually.
    var visualTree =
AssetDatabase.LoadAssetAtPath<VisualTreeAsset>("Assets/Editor/SimpleCustomEdi
tor_uxml.uxml");
    VisualElement labelFromUXML = visualTree.Instantiate();
    root.Add(labelFromUXML);
}
```

6.
7. Select **Window** > **UI Toolkit** > **SimpleCustomEditor** to open your custom Editor window to see three labels, three buttons, and three toggles.



Custom Editor Window with three Controls

Define the behavior of your UI controls

You can set up event handlers for your UI controls so that when you click the button, and select or clear the toggle, your UI controls perform some tasks.

In this example, set up event handlers that:

> When a button is clicked, the Editor Console displays a message.
> When a toggle is selected, the Console shows how many times the buttons have been clicked.

Your finished `SimpleCustomEditor.cs` looks like the following:

```
using UnityEditor;
using UnityEngine;
using UnityEngine.UIElements;

public class SimpleCustomEditor : EditorWindow
{
    [SerializeField]
    private VisualTreeAsset m_VisualTreeAsset = default;

    [MenuItem("Window/UI Toolkit/SimpleCustomEditor")]
    public static void ShowExample()
    {
        SimpleCustomEditor wnd = GetWindow<SimpleCustomEditor>();
        wnd.titleContent = new GUIContent("SimpleCustomEditor");
    }

    private int m_ClickCount = 0;

    private const string m_ButtonPrefix = "button";

    public void CreateGUI()
```

```
    {
        // Each editor window contains a root VisualElement object
        VisualElement root = rootVisualElement;

        // VisualElements objects can contain other VisualElement following a
tree hierarchy.
        Label label = new Label("These controls were created using C#
code.");
        root.Add(label);

        Button button = new Button();
        button.name = "button3";
        button.text = "This is button3.";
        root.Add(button);

        Toggle toggle = new Toggle();
        toggle.name = "toggle3";
        toggle.label = "Number?";
        root.Add(toggle);

        // Instantiate UXML created automatically which is set as the default
VisualTreeAsset.
        root.Add(m_VisualTreeAsset.Instantiate());

        // Import UXML created manually.
        var visualTree =
AssetDatabase.LoadAssetAtPath<VisualTreeAsset>("Assets/Editor/SimpleCustomEdi
tor_uxml.uxml");
        VisualElement labelFromUXML = visualTree.Instantiate();
        root.Add(labelFromUXML);

        //Call the event handler
        SetupButtonHandler();
    }

    //Functions as the event handlers for your button click and number counts
    private void SetupButtonHandler()
    {
        VisualElement root = rootVisualElement;

        var buttons = root.Query<Button>();
        buttons.ForEach(RegisterHandler);
    }

    private void RegisterHandler(Button button)
    {
        button.RegisterCallback<ClickEvent>(PrintClickMessage);
    }

    private void PrintClickMessage(ClickEvent evt)
    {
        VisualElement root = rootVisualElement;
```

```
        ++m_ClickCount;

        //Because of the names we gave the buttons and toggles, we can use
the
        //button name to find the toggle name.
        Button button = evt.currentTarget as Button;
        string buttonNumber = button.name.Substring(m_ButtonPrefix.Length);
        string toggleName = "toggle" + buttonNumber;
        Toggle toggle = root.Q<Toggle>(toggleName);

        Debug.Log("Button was clicked!" +
            (toggle.value ? " Count: " + m_ClickCount : ""));
    }
}
```

Test the example

From the menu, select **Window** > **UI Toolkit** > **SimpleCustomEditor**.

Additional resources

UXML
Visual Tree

Label
Button
Toggle

Create a custom Editor window

UI Builder

UI Builder

UI Builder lets you visually create and edit UI assets, such as UI Documents (`.uxml`), and StyleSheets (`.uss`), that you use with UI Toolkit.

You can also install the following optional packages that offer additional functionality in UI Builder:

The `com.unity.vectorgraphics` package allows you to assign a `VectorImage` as a background style on an element.
The `com.unity.2d.sprite` package allows you to assign a 2D **Sprite**

asset (or sub-asset) as a background style on an element. With the 2D Sprite package installed, you can also open the 2D Sprite Editor directly from the **Inspector**

pane.

Topic	Description
UI Builder interface overview	Understand the UI Builder interface.
Get started with UI Builder	Understand the workflow to create UI in UI Builder by an example.
Work with elements	Learn how to structure UI controls in your project.
Use UXML instances as templates	Learn how to use existing UXML Documents as templates inside your UXML Documents in UI Builder.
Style UI with UI Builder	Learn how to create and manage USS styles, how to use USS selectors and variables, and how to position elements and set background images.
Assign USS variables in UI Builder	Learn how to assign and remove a USS variable in UI Builder.
Test UI	Debug your styles and test inside the UI Builder.

Additional resources

Get started with UI Toolkit
Get started with runtime UI

Structure UI with UXML
Style UI with USS